TESTABLE WHITEBOX . TOOL VALIDATION AUTOMATION

Workflow Guide

How the **merge-flow** branch script works, file by file, and how to run it step by step

BRANCH	LATEST COMMIT	WRITTEN ON	STACK
merge-flow	`fd4e101`	29 Oct 2026	Playwright, TypeScript, Node 22, Python

What is inside

1 The big picture
what the script does in one minute

2 The pipeline
browser lane and background lane, side by side

3 Main flow
the 15 steps on Testable (including early blocked exit)

4 Background worker
hand-off, S3 check, multi-language validation, Teams

5 How to use it
one-time setup, daily checklist, dual-account parallel

6 File map
which file does which job (pages, tests, utils)

7 .env reference
every setting, what to put, defaults

8 Result folders
what each output file is (HTML, JSON, evidence)

9 Troubleshooting
symptom, cause, fix (AWS keys, blocked, freezes)

10 Cheat sheet & Queue
commands, words, freezeTracker, and keepAwake

Who this is for

Anyone who runs the script, reads its results, or needs to maintain it. You do not need to know the code to follow sections 1 to 5, 7, 8, 9, and 10. Section 6 is for engineers who inspect or modify the files. This guide is updated for the latest **merge-flow** branch containing deep Java, TypeScript, C#, and JavaScript validators, automated defect drafting, dual-account concurrency, and early blocked-run detection.

1

The big picture

what the script does in one minute

The script tests a branch on the Testable platform exactly like a person would, then checks what the platform wrote to AWS S3, and sends the result to the team. You give it one or more GitHub branch links. For every branch it does this:

• **Runs the branch on Testable with a real browser (Playwright):** connects the repository, picks the branch, presses Execute, waits for the analysis to finish, and takes screenshots and the taxonomy report.

• **Checks the evidence in S3:** for every technique it opens the tool's folder, reads the files, and says PASS, EMPTY or FAIL. Dedicated language validators (Python, Java, TypeScript, JavaScript, C#) judge whether each tool behaved correctly for that language and version.

• **Writes bug drafts (PDF, screenshot, Jira text):** automatically flags platform defects (missing tools, runner crashes, empty logs). Nothing is sent to Jira automatically without review.

• **Uploads the whole branch folder to SharePoint** and posts a card in Teams with the result, tool summary, and link.

BROWSER LANE (one branch at a time)	BACKGROUND LANE (no browser, runs beside next branch)
You start it .env + builds.json or BRANCHES	**S3 check** tool folders, validators, bug drafts, report
Login (auth.setup.ts) saves session to .auth/user.json	**SharePoint** whole branch folder uploaded (date/branch)
Main flow steps 1 to 15 on Testable (Playwright)	**Teams card** result + evidence link + tool summary
Hand-off job file written, next branch starts at once	**Next job** or finish when queue is empty

The key idea of this branch: after the browser part of a branch is finished, everything that does not need the browser (S3 check, SharePoint, Teams) moves to a background worker. The next branch starts on the platform right away, so the platform is never waiting for your reports to be written.

Three things to remember

1. The browser lane runs one branch at a time.
2. The background lane works on branch N while branch N+1 is executing.
3. Every setting lives in **.env** (section 7). A change to .env always wins over a value typed in the terminal.

2

The pipeline

why the platform no longer sits idle

Before, the script finished all the reports of a branch before it started the next one, so the platform waited. Now the reports are made by the background worker beside the browser:

Model	Execution Timeline	Platform Efficiency
BEFORE (inline)	`Branch 1 [10m] -> S3 & Reports [4m] -> Branch 2 [10m] -> S3 & Reports [4m]`	**~30% Idle Time** Platform waits for reports
NOW (merge-flow worker)	`Browser: Branch 1 [10m] -> Branch 2 [10m] -> Branch 3 [10m] -> Logout` `Worker: post 1 [4m] -> post 2 [4m] -> post 3 [4m]`	**100% Efficiency** Zero platform idle gap

The order of every run

#	Moment	What happens	Where you see it
1	Start	Branches are read from BRANCHES or builds.json. The run folder results/merge-flow__ is created.	Terminal / Test Explorer
2	Login	auth.setup.ts logs in with Testable credentials from .env and saves session in .auth/user.json.	`test 'auth.setup.ts'`
3	Branch N (Browser)	Main flow steps 1 to 15: connect repo, execute, wait, capture score, download taxonomy report.	`test 'Whitebox run - '`
4	Hand-off	When branch N passes, a job file is written to .postprocess/ and worker starts. Branch N+1 starts at once.	`line 'handed to background worker'`
5	Branch N (Worker)	S3 check, multi-language validation, bug drafts, SharePoint upload, Teams card. Runs beside branch N+1.	`lines '[post] [branch] ...'`
6	Logout	After the last branch finishes execution, the browser logs out of Testable.	`test 'Logout'`
7	Worker Finish	The final branch's post-processing completes; worker closes. The final verification test turns green.	`test 'Background finishes'`

3

Main flow

the 15 steps the browser does for each branch

Steps 1-2: Start Open chat, New analysis (first branch only)	**Steps 3-7: Pick Branch** Connect repo, pick branch, tick 'On PR merge', confirm
Steps 8-9: Preview Coverage preview, language read, **early exit if blocked**	**Steps 10-12: Execute & Wait** Execute run, open Confidence Engine, wait up to 35 min
Steps 13-15: Collect Score screenshot, gate values, taxonomy report download	**Hand-Off / Branch End** Passed -> Worker queue; Blocked/Failed -> Teams alert

Detailed Step Inventory & Logic in MergeFlowPage.ts

Step	Action & Logic in MergeFlowPage.ts	Critical?	Saved Output
0	Optional: with DUMMY_PR=yes, pushes a comment to branch README so platform sees fresh commit.	No	Git commit on remote
1	Opens Chat Execution Assistant (later branches use sidebar chat icon).	**Critical**	`evidence/_1_PASS.png`
2	First branch only: 'Can I safely deploy this commit?' -> New analysis / Refresh repos.	No	-
3	Change repository / branch -> Connect / Import repository. Waits for repo sync (cap 14m).	**Critical**	-
4	Clicks repository card -> Configure repository.	**Critical**	-
5	Selects branch: '+N branches', searches branch name, ticks checkbox, clicks Done.	**Critical**	-
6	Makes sure the 'On PR merge' trigger checkbox is ticked.	**Critical**	evidence screenshot
7	Confirms repository. If platform answers 'Branch unavailable', marks branch BLOCKED.	**Critical**	`errors/site-error.txt`
8	Coverage preview: repo structure, what will be analyzed. **NEW: waitSectionOrBlocked stops after 10s if blocked.**	No	`01_, 02_, 03_ shots, repo-details.json`
9	Continues from preview / Review branches. Stops if site blocks analysis.	**Critical**	-
10	Run plan -> Execute run. Retries if run does not trigger immediately.	**Critical**	evidence screenshot
11	Execution status -> Open Confidence Engine.	**Critical**	-
12	Waits for analysis to finish (polls green checkmark, default cap 35 min).	**Critical**	`05_execution_status.png`
13	Captures full screenshot of PR Confidence Analysis score.	No	`04_pr_confidence_score.png`
14	Reads every technique x classification gate value from UI. With GATE_SHOTS=on takes gate shots.	No	`ui-values.json, gates/`
15	Downloads taxonomy report HTML, saves run-details.json (Run ID, tenant, commit, branch), merges UI values.	No	`taxonomy-gate-*.html, run-details.json`

When a branch does not pass

how failures, blocks, and environment errors are handled safely

A failed, blocked, or environment problem ends **that branch only**. A failure screenshot and network trace are saved in errors/, a failure alert card goes to Teams immediately, and the next branch starts. A branch that failed has no background post-processing because no platform run was produced.

Status Code	Trigger Condition	Automated System Response
BLOCKED	Site says 'Analysis is blocked on ...' or 'Branch unavailable'.	Stops in 10s via waitSectionOrBlocked. Saves errors/site-error.txt. Skips to next branch.
ENVIRONMENT	Site down, 502/504 Bad Gateway, repository sync hung.	Captures failure.png and network.json. Sends Teams failure card. Proceeds to next branch.
FAILED	Critical UI step timed out or button locator not found.	Saves error trace in errors/. Sends Teams card with failure reason screenshot. Moves to next branch.

4

Background worker

what happens to a passed branch after the browser is done

4.1 The Hand-Off (.postprocess/)

The browser test and the worker communicate through atomic state files in results//.postprocess/. Nothing is lost if either process stops or restarts:

File in .postprocess/	Operational Meaning & Lifecycle Role
`-.job.json`	A branch waiting to be post-processed by the worker.
`...job.json.running`	Currently being processed. If the worker crashes, it is automatically renamed back to .job.json so the next worker retries it.
`...done.json / ...failed.json`	Successfully completed or permanently failed jobs.
`worker.pid, CLOSE, worker.done`	Worker process registration, 'all branches sent', and 'worker finished'.

4.2 What the Worker Does for Each Branch

Order	Step	Details & Responsibilities	If Failed
1	S3 Check	Reads AWS S3 tool folders, runs multi-language validators, writes s3-check.json and bug drafts.	Logged; SharePoint and Teams still run
2	SharePoint	Uploads the complete branch directory to team library (Automation Evidences Dump/date/branch).	Logged; Teams still runs
3	Teams	Posts success notification card with direct run link, SharePoint evidence link, and tool breakdown.	Marked failed; redo with npm run post

4.3 The S3 check, explained

how tool trigger status and multi-language logs are evaluated

Every tool executed on the platform writes its artifacts to one dedicated S3 path built from the run record: development/cell-001/<tenant>/<project>/<repository>/<branch_id>/<branch>/<commit>/<run_id>/<tool>/...

The 8 Steps of the S3 Check Pipeline

1. Find Run: run-details.json gives tenant/repo/branch/run ID.	**2. List Folders:** scans S3 prefix -> s3-folders.json.
3. Pick Language: coverage preview or branch prefix (JS_, TS_, PY_, CS_, JV_).	**4. Technique > Tool:** techniqueTools.ts resolves tool name candidates.
5. Read Files: downloads JSON/XML tool files to s3-evidence/.	**6. Assign Verdict:** PASS (data), EMPTY (0 findings), FAIL (missing/error).
7. Language Validator: deep checks for version compatibility & false empties.	**8. Generate Reports:** writes s3-check.json and bug drafts in bugs/.

Tool Verdicts Reference

Verdict	Meaning	Typical Reason / Evidence
PASS	The tool was triggered and produced valid output data.	e.g. lizard.xml with 123 functions, bandit.json with findings.
EMPTY	Triggered, but report holds 0 findings or was marked not applicable.	status=not_applicable, baseline_missing, clean code (0 defects).
FAIL	Not triggered, missing tool folder, 0 bytes, or failure.json.	No folder in S3, [permanent_runner_error], container crash.
N/A	The tool does not apply to this specific taxonomy row.	Primary tool not applicable for this classification level.

The 5 Dedicated Multi-Language Validators

Language	Validator File	Evaluated Tools & Version Checks
Java	`utils/javaValidate.ts`	JaCoCo, SpotBugs, Checkstyle, PMD, PIT mutation testing across JDK 8, 11, 17, 21 and Maven/Gradle.
TypeScript	`utils/tsValidate.ts`	ESLint, TSC, JSCPD, Jest/Vitest coverage, Snyk across Node 18, 20, 22 and Webpack/Esbuild/Vite.
JavaScript	`utils/jsValidate.ts`	ESLint, JSCPD, Mocha/Istanbul, Retire.js across npm/yarn/pnpm mono and micro repos.
C# (.NET)	`utils/csValidate.ts`	Roslyn Analyzers, StyleCop, Coverlet/dotCover, SecurityCodeScan across .NET 6, 7, 8, 10.
Python	`utils/pyValidate.ts`	Coverage.py, Bandit, Pylint, Flake8, Lizard, Radon across Python 3.7 to 3.14.

Bug drafts, SharePoint & Teams

automated defect reporting and team distribution

4.3 Automated Bug Drafts (utils/bugReport.ts)

When the language validators detect a platform problem, bugReport.ts automatically generates a complete defect package into results/<run>/<branch>/bugs/ without manual intervention:

• **Defect Classification:** Classifies issues into [Planning Defect] (tool missing from plan), [False Empty Defect] (tool reported 0 findings after scanning 0 files), [Runner Execution Defect] (container error/crash), or [Language Contamination] (wrong language tool triggered).

• **Executive PDF Bug Report:** A clean 2-page PDF report detailing the exact failure, S3 path, tool parameters, and error excerpt.

• **Visual Screenshot:** Captures a targeted crop of the taxonomy report row showing the failure.

• **Jira Text File:** Pre-formats a ready-to-paste Jira ticket description with reproduction steps and native attachment links.

4.4 SharePoint Upload & _RUN Naming

Uploads the entire branch folder to **Automation Evidences Dump** on the team site. It uses a tiered connection strategy:

Priority	Method	Configuration & Behavior
1 (Best)	Microsoft Graph API	Requires SP_CLIENT_ID and SP_TENANT_ID in .env. Unattended token refresh.
2 (Easiest)	Personal Session	Run npm run sp:login once. Uses saved browser session in .auth/. No admin required.
3 (Fallback)	OneDrive Shortcut	Requires SHAREPOINT_DIR path. Copies to local synced OneDrive directory.

Note: Multiple runs of the same branch on the same day automatically append _RUN2, _RUN3 to avoid overwriting earlier runs.

4.5 Microsoft Teams Integration

Notification Type	Timing	Card Contents & Actions
Success Card	Sent by background worker after S3 & SharePoint	Branch name, repo, QA/DEV environment, execution duration; direct buttons: 'Open Run on Site' and 'Open SharePoint Evidence'; tool breakdown summary.
Failure Card	Sent by browser test immediately upon failure	Branch name, failure classification (failed / blocked / environment), exact error message, and failure screenshot attachment.
Queue Abort Card	Sent by queue server if round stalls	Alerts team if login or platform site is unreachable before a branch can begin.

5

How to use it

one-time setup, then the same checklist every run

5.1 One-Time Setup (Per Machine)

#	Action Item	Command or Location
1	Install Node.js 22 LTS, Git, and Python 3.11+. Clone repository and switch to merge-flow.	`git checkout merge-flow`
2	Install Node dependencies (Playwright, AWS SDK, exceljs, tsx).	`npm install`
3	Install the Playwright Chromium browser binaries.	`npx playwright install chromium`
4	Copy .env.example to .env and configure credentials (section 7).	`Copy-Item .env.example .env`
5	Sign in to SharePoint once with your Microsoft work account.	`npm run sp:login`
6	Verify Python virtual environment (if running CLI tools or EKS mentor).	`pip install -r requirements.txt`

5.2 Daily Pre-Run Checklist

#	Step	Why It Matters
1	Refresh AWS keys in .env (AWS IAM Identity Center -> QA-S3-Readonly -> Access keys).	AWS temporary credentials expire after several hours; avoids ExpiredToken error.
2	Put target branch URLs into builds.json or set $env:BRANCHES in PowerShell.	Defines the execution queue for the runner.
3	Start the test run. A Chromium window opens. **Do not minimize or lock screen.**	The headed browser needs active viewport rendering for screenshots.
4	Do not edit .env or config files while the run is active.	Modifying configs triggers a hot reload and terminates active sessions.
5	Monitor terminal progress. Look for [post] [branch] output lines.	Confirms the background worker is processing S3 artifacts concurrently.
6	When 'Background post-processing finishes' turns green, execution is complete.	All S3 checks, SharePoint uploads, and Teams cards are finished.
7	Run git checkout builds.json to keep your git working tree clean.	Prevents personal test branch URLs from accidentally being committed.

5.3 Two ways to start a run

terminal commands and test explorer workflows

Method A: PowerShell Terminal	Method B: VS Code / Antigravity Test Explorer
Set target branch URLs and run: $env:BRANCHES="https://github.com/OWNER/REPO/tree/BRANCH1,https://github.com/OWNER/REPO/tree/BRANCH2" npm test *Multiple branches can be comma-separated. The repo is inferred directly from the GitHub link.*	1. Open builds.json and paste branch links. 2. Click the Refresh icon in the Testing panel. 3. Ensure **Show browser** is toggled on. 4. Run the entire **flow** project (do not run individual step tests singly).

5.4 Commands Reference Table

Command	Purpose & Operational Description
`npm test`	Executes the complete flow: login, all branches in builds.json, logout, and background worker drain.
`npm run setup`	Only performs the Testable login and saves session to .auth/user.json. Use to verify credentials.
`npm run sp:login`	Launches interactive Microsoft browser login for SharePoint upload access token.
`npm run s3`	Standalone S3 audit of an existing results directory without re-running the browser UI.
`npm run post -- "<dir>"`	Re-runs post-processing (S3 check, bug drafts, SharePoint, Teams) for a specific finished branch folder.
`npm run queue`	Starts the local Express queue management server on http://localhost:3000.
`npm run parallel`	Executes dual-account concurrent lanes (Account 1 and Account 2) simultaneously on one PC.

5.5 Two Testable Accounts on One PC (Parallel Lanes)

To run two accounts concurrently on the same machine without collisions, configure TESTABLE_EMAIL_2 and TESTABLE_PASSWORD_2 in .env. Then run npm run parallel. Account 2 uses its own isolated storage state (.auth/user-2.json) and results folder (..._acct2). Work is dynamically distributed across both accounts.

6

File map

which file does which job in the repository

The system is organized into decoupled functional layers. The table below documents every file in the root, pages, and tests directories:

File / Path	Function & Responsibility
Project Root	**Configuration & Environment**
`package.json`	Defines project metadata, dependencies (Playwright, AWS SDK, exceljs, tsx), and npm runner scripts.
`playwright.config.ts`	Main Playwright configuration. Configures headed Chromium, 1 worker, 35m timeouts, device scale factor, and test projects (setup, flow, s3).
`tsconfig.json`	TypeScript compiler configuration enforcing strict type checking and modern module resolution.
`builds.json`	Input JSON array of target GitHub branch URLs. Rewritten by queue server during automated rounds.
`.env / .env.example`	Central secrets file: Testable login, AWS access keys, Teams webhook, SharePoint targets. Never committed.
`.gitignore`	Prevents secrets (.env), sessions (.auth/), test results (results/), and cache from entering git.
pages/	**UI Interaction & Selectors**
`MergeFlowPage.ts`	Comprehensive Page Object encapsulating all 15 Testable UI steps. Contains self-healing locators, chat assistant handling, coverage preview inspection, Confidence Engine polling, and early blocked-run detection (waitSectionOrBlocked).
tests/	**Test Runners & Specs**
`auth.setup.ts`	Initial authentication test. Logs into Testable (or Okta SSO) and writes persistent browser state to .auth/user.json.
`merge-flow.spec.ts`	Main branch execution spec. Iterates through builds.json, executes steps 1-15, saves network traces on failure, and hands off jobs to the background queue.
`s3-evidence.spec.ts`	Standalone S3 evidence verification spec for re-evaluating historical results folders without opening the browser.

6.2 utils/ and queue-server/ files

the backend engines for S3, validation, reporting, and queue management

File in utils/	Detailed Responsibility & Role
awsS3.ts	AWS S3 client wrapper handling authentication, prefix listing, exponential backoff retries, and token expiry detection.
s3Evidence.ts	Core S3 evidence scanner. Builds inventory, matches candidate tools, downloads logs, and assigns PASS/EMPTY/FAIL verdicts.
s3Check.ts	Inspects individual tool JSON/XML log contents, verifies structure, and detects programming languages.
techniqueTools.ts	Master catalog dictionary defining primary and secondary tool mappings per testing type and technique across languages.
mergeTaxonomy.ts	Merges S3 tool validation columns into the platform's HTML taxonomy report (*-with-ui-values.html).
pyValidate.ts	Deep validator for Python tools (Coverage.py, Bandit, Pylint, Lizard, Radon) checking version-specific rules.
javaValidate.ts	Deep validator for Java tools (JaCoCo, SpotBugs, Checkstyle, PMD, PIT) across JDK versions and build tools.
tsValidate.ts / jsValidate.ts	Deep validators for TypeScript and JavaScript tools (ESLint, TSC, JSCPD, Jest/Vitest, Retire.js).
csValidate.ts	Deep validator for C# (.NET) tools (Roslyn Analyzers, StyleCop, Coverlet/dotCover).
bugReport.ts	Defect reporting engine that builds 2-page PDF bug reports, row screenshots, and Jira Markdown bodies.
postprocess*.ts	Asynchronous worker queue implementing file-based job scheduling (.postprocess/) and worker lifecycle.
sharepoint*.ts / spLogin.ts	SharePoint client supporting Microsoft Graph API, interactive personal sign-in, and OneDrive shortcut copies.
teams.ts / teamsWorker.js	Constructs and dispatches rich adaptive cards to Microsoft Teams webhooks with tool summary tables.
parallelRun.ts	Concurrent scheduler managing dual-account execution lanes without test artifact collisions.
queue-server/	**Local Queue Dashboard & Reliability Engines**
server.js	Express server on Port 3000 providing web dashboard and REST API for team branch queuing.
freezeTracker.js	Watchdog monitoring branch startup. Detects inactivity > 8 minutes and restarts frozen runs automatically.
keepAwake.js	Prevents Windows OS sleep/idle suspension during long overnight batch validation runs.

7

.env settings reference

every configuration variable, what to put, and defaults

Variable	Required?	Meaning & Valid Values	Default
TEST_ENV	Yes	Platform environment to target: qa (qa-frontend.testable.cc) or dev.	qa
TESTABLE_EMAIL	Yes	Login email address for the Testable platform account.	-
TESTABLE_PASSWORD	Yes	Password for the primary Testable platform account.	-
TESTABLE_EMAIL_DEV	For dev	Login email for dev environment (used when TEST_ENV=dev).	-
AWS_ACCESS_KEY_ID	For S3	Temporary AWS Access Key ID from IAM Identity Center (QA-S3-Readonly).	-
AWS_SECRET_ACCESS_KEY	For S3	Temporary AWS Secret Access Key.	-
AWS_SESSION_TOKEN	For S3	Temporary AWS Session Token (must be refreshed when expired).	-
AWS_REGION	Yes	AWS region where evidence S3 bucket resides.	us-east-2
TEAMS_WEBHOOK_URL	Optional	Microsoft Teams workflow webhook URL. Empty = no Teams alerts sent.	empty
POSTPROCESS	Optional	background = run S3/reports in background worker; inline = sequential.	background
DUMMY_PR	Optional	yes = push comment to branch README before run to force fresh commit.	no
GATE_SHOTS	Optional	on = screenshot every gate; off = only extract gate score values.	off
RUN_TIMEOUT_MIN	Optional	Maximum minutes to wait for analysis to complete on platform.	35
SP_TENANT_ID / CLIENT_ID	Optional	Microsoft Graph App Registration for unattended SharePoint upload.	-
SHAREPOINT_DIR	Optional	Local OneDrive synced path for team evidence folder upload.	-
TESTABLE_EMAIL_2	Parallel	Second account email for dual-account concurrent execution (npm run parallel).	-

8

Result folders

what each output file is in results/merge-flow_/

Every execution creates a dedicated run directory under results/merge-flow_<date>_<time>/. Inside, each branch has its own complete subfolder containing all evidence:

Path / Artifact Name	Artifact Content & Functional Purpose
`taxonomy-gate-*.html`	Official HTML gate report downloaded directly from Testable platform.
`*-with-ui-values.html`	Enhanced HTML report merging UI gate scores with AWS S3 tool execution columns.
`run-details.json`	JSON record of the run: tenant_id, project_id, branch_id, commit_sha, and run_id.
`repo-details.json`	Language, build type, and framework detected during the coverage preview.
`ui-values.json`	Raw key-value pairs of every technique and classification score read from the UI.
`s3-check.json`	Complete verdict manifest: every tool evaluated with PASS / EMPTY / FAIL and note.
`s3-folders.json`	List of all directories created by tools in the AWS S3 run prefix.
`s3-files.json`	Heads and excerpts of all JSON and XML files found in S3.
`s3-evidence/`	Local directory containing all downloaded raw tool output files (logs, XML, JSON).
`bugs/`	Generated defect drafts: 2-page PDF bug reports, failure row crops, and Jira text.
`errors/`	Captured failure.png screenshots, network.json traces, and site-error.txt (if blocked).
`postprocess.json`	Background worker status tracker: records state of S3, SharePoint, and Teams jobs.
`postprocess.log`	Complete console log of the background post-processing worker.

9

Troubleshooting

symptom, root cause, and immediate resolution

Observed Symptom	Likely Root Cause	Immediate Fix Procedure
AWS keys expired / S3 check skipped	AWS temporary session tokens expired.	Copy fresh keys from AWS Portal into .env. Redo check via: npm run post -- "<folder>".
Branch stops: BLOCKED by site	Site says 'Analysis is blocked on ...' or commit already tested.	Normal behavior: waitSectionOrBlocked exits in 10s. Set DUMMY_PR=yes in .env to push fresh commit.
Browser Zoom / Layout Shift	Windows OS scaling (125% / 150%) shifts button coordinates.	Config sets --force-device-scale-factor=1. For custom scale: $env:PAGE_SCALE="0.9".
Runner freezes at start of branch	Platform navigation stalls or network handshake hangs.	The built-in freezeTracker.js automatically detects inactivity > 8m and restarts the queue safely.
Machine sleeps during overnight run	Windows power-saving policy puts machine to sleep.	Run queue via npm run queue. It runs keepAwake.js issuing power assertions to prevent sleep.
RUN-MISMATCH.txt in branch folder	The run opened on the site belongs to another branch.	Do not trust that folder. Run the branch again to ensure correct association.
A terminal variable is ignored	.env file values always win over terminal environment variables.	Edit the variable directly in .env instead of setting it in the terminal.

First place to look when debugging

For any problem after a branch finishes: open <branch folder>/postprocess.json. It tells you which of S3, SharePoint, and Teams succeeded or failed and shows the exact error message. The full log is in postprocess.log.

10

Cheat sheet

commands, terminology, and the team queue server

Words Used in This Guide

Term	Definition
Branch folder	results/<run>/<branch_name>/ : houses all reports, screenshots, and evidence for one branch.
Main flow / Browser lane	The 15 sequential UI steps executed in Playwright Chromium on the Testable site.
Worker / Background lane	The decoupled Node.js process performing S3 checks, validation, SharePoint uploads, and Teams cards.
Hand-off	The moment a passed branch writes a job file to .postprocess/ and allows the next branch to start.
Platform defect	A bug belonging to Testable (missing tool, runner crash, false empty) rather than the user's code.
Blocked / Environment / Failed	The 3 exit statuses: platform rejected run, network/site unavailable, or UI step assertion failed.

The Team Branch Queue Server (npm run queue)

When multiple team members submit validation branches, launch npm run queue and open http://localhost:3000. Engineers enter their branch links and name. The server owner types start in the terminal. The queue serves one branch from each waiting person in a fair round-robin order. It incorporates freezeTracker.js to auto-recover stalled runs and keepAwake.js to prevent system sleep.

About This Document

This manual describes the **merge-flow** branch at commit fd4e101. When code or selector logic in pages/MergeFlowPage.ts or utils/ is updated, update the corresponding section in this guide. The automation logs its operations in real time; when in doubt, trust the live logs and postprocess.json.